

Esgrima en la era prehistórica

Informe del Proyecto Final

Jeyson Sarrazola Blanco

Elías Luna Pérez

Informática II 2026-1

Anibal Guerra Soler

1. Introducción

Esgrima en la era prehistórica es un videojuego desarrollado en C++ con el framework Qt como proyecto final del curso de informática II. El juego combina la esgrima con lanzas de obsidiana en un entorno prehistórico volcánico, donde el jugador enfrenta tanto los peligros del ambiente como a un rival con comportamiento inteligente.

El proyecto cuenta con dos niveles de juego con mecánicas completamente distintas. En el Nivel 1 el jugador destruye rocas volcánicas que crecen con el tiempo y que le hacen daño al usuario si colisiona ante ellas, mientras lidia con temblores sísmicos. En el Nivel 2 se enfrenta en un duelo de esgrima 1 contra 1 contra un agente autónomo que aprende y adapta su estrategia a lo largo del combate.

El proyecto fue desarrollado en paralelo por dos integrantes: uno a cargo del Nivel 1 y los menús, y otro a cargo del Nivel 2 y el agente autónomo, integrando ambas partes en una rama dedicada al final del desarrollo.

2. Objetivos

2.1 Objetivo General

Desarrollar un videojuego de dos niveles en C++ con Qt que demuestre la aplicación de programación orientada a objetos, manejo de memoria dinámica, interfaces gráficas y un agente autónomo con comportamiento adaptativo.

2.2 Objetivos Específicos

- Construir una jerarquía de clases propia con herencia y polimorfismo.
- Aplicar el patrón Strategy para gestionar la dificultad del Nivel 1 de forma flexible.
- Diseñar un agente autónomo con ciclo completo: percepción, razonamiento, acción y aprendizaje.
- Mantener la lógica del juego completamente separada de la capa gráfica de Qt.
- Integrar audio, animaciones y sprites para enriquecer la experiencia del jugador.

3. Especificación de Requerimientos

3.1 Requerimientos Funcionales

ID	Nombre	Descripción
01	Menú principal	Muestra opciones: Nivel 1, Nivel 2 y Salir.
02	Selección de dificultad	El jugador elige entre Fácil y Difícil antes del Nivel 1.

ID	Nombre	Descripción
03	Control N1	El jugador se mueve en 8 direcciones y ataca rocas en rango.
04	Generación de rocas	Las rocas crecen desde el suelo y desaparecen si no se destruyen.
05	Sistema de vidas	El jugador tiene 3 vidas; pierde una al colisionar con una roca.
06	Temblor sísmico N1	A intervalos periódicos el escenario tiembla con oscilación amortiguada.
07	Victoria N1	El nivel termina exitosamente al destruir el objetivo de rocas antes de que se agote el tiempo.
08	Combate N2	Dos combatientes ejecutan ataques, saltos y esquivas en plataforma 2D lateral.
09	Agente autónomo	El enemigo percibe, razona, actúa y aprende del jugador en cada ronda.
10	Temblor sísmico N2	A intervalos aleatorios ambos personajes se desplazan con la misma física del N1.
11	Transiciones	El juego muestra pantallas de inicio de ronda, victoria y derrota con sonidos.
12	Volver al menú	El jugador puede regresar al menú principal pulsando P en cualquier momento.

3.2 Requerimientos No Funcionales

ID	Categoría	Descripción
01	Rendimiento	Game loop a ~60 FPS mediante QTimer a 16 ms.
02	Portabilidad	C++ con Qt.
03	Mantenibilidad	La lógica del juego no depende de Qt.
04	Usabilidad	Controles estándar WASD + Shift + Espacio fáciles de aprender.
05	Extensibilidad	El patrón Strategy permite agregar dificultades sin modificar clases existentes.

3.3 Restricciones

- Lenguaje: C++.
- Framework: Qt (Widgets, Multimedia).
- Sistema de construcción: qmake (.pro).
- Resolución fija de ventana: 900 × 700 píxeles.

- Sin persistencia en disco: el estado del agente se reinicia al salir.

4. Metodología y Planificación

El desarrollo se organizó de forma iterativa con dos ramas de trabajo paralelas en Git. Un integrante se encargó del Nivel 1 y los menús (rama Nivel1), y el otro del Nivel 2 con el agente autónomo (rama Nivel2). La integración se realizó en una tercera rama dedicada (integracion), y el merge final al main lo ejecutó el responsable de integración.

Fases del Desarrollo

- Fase 1: Análisis y diseño. Definición de jerarquías, patrones y responsabilidades de cada clase.
- Fase 2: Desarrollo paralelo. Nivel 1 + menús en una rama; Nivel 2 + agente en otra.
- Fase 3: Integración, ajuste de MainWindow y resolución de conflictos.
- Fase 4: Pruebas y corrección. Verificación manual de cada requerimiento funcional.
- Fase 5: Elaboración del archivo ejecutable.
- Fase 6: Documentación. Elaboración del presente informe.
- Fase 7: Elaboración del diagrama de clases.
- Fase 8: Elaboración del tráiler del videojuego.
- Fase 9: Elaboración del video explicativo del proyecto.

5. Diseño y Arquitectura del Sistema

5.1 Visión General

El sistema se divide en dos capas: la capa lógica, que contiene todas las reglas del juego en C++ puro sin depender de Qt, y la capa gráfica, que usa Qt para renderizar, capturar input y reproducir audio. La comunicación entre capas va en un solo sentido: la GUI lee el estado de la lógica y lo dibuja.

La ventana principal usa un QStackedWidget para alternar entre el Nivel 1 (QGraphicsScene) y el Nivel 2 (QWidget con paintEvent manual), sin necesidad de crear o destruir ventanas al cambiar de nivel.

5.2 Diagrama de Clases

El diagrama resumen se encuentra en la carpeta de Informes, el diagrama completo al ser tan extenso decidimos mejor mostrarlo en el video explicativo del funcionamiento del programa.

5.3 Por qué Nivel 1 y Nivel 2 son clases separadas

Aunque ambos niveles comparten el mismo sentido (ser un nivel), sus mecánicas son tan distintas que unirlos en una sola clase habría generado una clase enorme y difícil de mantener. El Nivel 1 maneja una colección dinámica de rocas, un sistema de vidas, un timer de 60 segundos y una vista cenital. El Nivel 2 gestiona dos combatientes, un sistema de puntos por rondas, detección de colisiones por embestida y una vista lateral con gravedad.

Mantenerlos como clases separadas permite que cada una tenga exactamente los atributos y métodos que necesita, que sean fáciles de probar de forma independiente, y que el código de integración (MainWindow) pueda manejarlos de forma uniforme aunque internamente sean completamente distintos.

5.4 Por qué Personaje y Combatiente son clases separadas

Durante el desarrollo, cada integrante creó una clase base para su personaje y ambos la llamaron Personaje. Sin embargo sus responsabilidades son radicalmente distintas: la clase del Nivel 1 es una entidad simple con posición, velocidad y sistema de vidas, mientras que la del Nivel 2 es un sistema complejo de combate con embestida física, salto con gravedad, esquite con impulso amortiguado, cooldowns de ataque y retroceso escalonado.

Unificarlas habría forzado a la entidad simple del Nivel 1 a cargar con toda la complejidad de combate que no usa, violando el principio de responsabilidad única. Al renombrar la del Nivel 2 a Combatiente se eliminó también el conflicto de nombres en el compilador, ya que ambos archivos coexisten en el mismo proyecto.

5.5 Jerarquía de Clases

Clase	Módulo	Responsabilidad
Entidad	Nivel 1	Base abstracta: posición, estado activo y actualizar().
Personaje : Entidad	Nivel 1	Agrega velocidad y sistema de vidas.
Jugador : Personaje	Nivel 1	Procesa input de teclado y limita posición al escenario.
Roca : Entidad	Nivel 1	Crece con el tiempo; se destruye por impacto o al alcanzar tamaño máximo.
Combatiente	Nivel 2	Base del combate: embestida, salto, esquiva, retroceso y gravedad.
Principal : Combatiente	Nivel 2	Lee el InputManager y delega en los métodos de Combatiente.
Enemigo : Combatiente	Nivel 2	Agente autónomo con ciclo percibir-razonar-actuar-aprender.
Temblor	Compartida	Física sinusoidal amortiguada; usada por ambos niveles.

Clase	Módulo	Responsabilidad
Dificultad (interfaz)	Nivel 1	Contrato de parámetros del juego (patrón Strategy).
DificultadFacil / Dificil	Nivel 1	Implementaciones concretas de Dificultad.
Nivel1	Nivel 1	Orquesta jugador, rocas, temblor y condiciones de victoria/derrota.
Nivel_2	Nivel 2	Orquesta combate, detección de toques, rondas y temblor.
EscenaNivel1	Vista	QGraphicsScene: sprites, HUD y timers del Nivel 1.
GameWidget	Vista	QWidget con paintEvent manual para el Nivel 2.
MainWindow	Ventana	QStackedWidget que alterna entre QGraphicsView y GameWidget.
EscenaMenu / Dificultad / Transicion	Vista	Pantallas de navegación con botones y audio.
SoundManager	Audio	Centraliza QMediaPlayer y QAudioOutput del Nivel 2.

5.6 Sistema de Dificultad con Polimorfismo

La dificultad del Nivel 1 se gestiona mediante el patrón Strategy. La clase Dificultad es una interfaz abstracta con métodos virtuales puros que definen los parámetros del juego. Las clases DificultadFacil y DificultadDificil la implementan con valores distintos. Nivel1 recibe un puntero a Dificultad y opera sobre él sin saber concretamente cuál es:

```

dif    = esDificil ? (Dificultad*) new DificultadDificil()
      |           |           | : (Dificultad*) new DificultadFacil();
nivel = new Nivel1(dif, 60);

```

Este diseño permite agregar nuevas dificultades sin modificar ninguna línea de Nivel1. Los parámetros que diferencian ambas dificultades son:

Parámetro	Fácil	Difícil	Efecto en el juego
Velocidad de crecimiento de roca	14 px/s	26 px/s	Las rocas alcanzan su tamaño máximo casi el doble de rápido.
Intervalo de spawn	2200 ms	1200 ms	Las rocas aparecen con el doble de frecuencia.
Máximo de rocas simultáneas	3	6	Hay el doble de amenazas activas en pantalla.

Parámetro	Fácil	Difícil	Efecto en el juego
Rocas objetivo para ganar	10	18	El jugador debe destruir casi el doble de rocas.
Intervalo entre temblores	12000 ms	6000 ms	Los temblores ocurren con el doble de frecuencia.
Amplitud del temblor	7 px	16 px	Los temblores desplazan el escenario más del doble.

5.8 Dependencias Externas

- Qt 6.x: módulos QtWidgets, QtMultimedia y QtCore.
- C++ Standard Library: <vector>, <cmath>, <algorithm>, <stdexcept>.
- Recursos embebidos: imágenes .png y audio .mp3 gestionados a través del sistema .qrc de Qt.

6. Desarrollo e Implementación

6.1 Físicas Implementadas

A continuación se describen todos los modelos físicos implementados en el proyecto. El movimiento rectilíneo uniforme no se incluye como física ya que el enunciado lo excluye explícitamente.

Física 1 — Crecimiento lineal de las rocas (Nivel 1)

Las rocas no aparecen con su tamaño final sino que crecen desde el suelo de forma progresiva, simulando una erupción volcánica. El modelo es lineal y parametrizable:

$$\text{tamaño}(t) = \text{tamañoInicial} + \text{velCrecimiento} \times t$$

Al alcanzar tamañoMax la roca desaparece sin ser destruida (el jugador no puntúa). La velocidad de crecimiento es el parámetro más visible que diferencia la dificultad Fácil (14 px/s) de la Difícil (26 px/s).

Física 2 — Temblor sísmico: oscilación sinusoidal amortiguada (Niveles 1 y 2)

Los temblores aplican un desplazamiento variable al escenario siguiendo la ecuación de un sismo real: oscila fuerte al inicio y decae progresivamente. La fórmula es:

$$\text{offset}(t) = A \cdot \sin(\omega \cdot t) \cdot e^{(-\lambda \cdot t)}$$

Los tres parámetros (A , ω , λ) son configurables, lo que permite que el Nivel 1 tenga un temblor más sutil ($A=7$ en fácil, $A=16$ en difícil) y el Nivel 2 uno más dramático ($A=18$). En el Nivel 1 el offset se aplica al renderizado del jugador y las rocas; en el Nivel 2 se aplica directamente a la posición de ambos combatientes.

Física 3 — Detección de colisión por distancia y producto punto (Nivel 1)

El ataque del jugador no es omnidireccional: solo golpea rocas que estén dentro del rango de ataque Y en la dirección en que mira el jugador. La detección usa dos condiciones:

- Distancia euclidiana $\leq$ radio de la roca + RANGO_ATAQUE (70 px): la roca está suficientemente cerca.
- Producto punto entre (dirección al centro de la roca) y (dirección que mira el jugador) > 0 : la roca está al frente.

Este modelo simula fielmente la mecánica de esgrima: solo se golpea lo que está frente al arma, no lo que está detrás.

Física 4 — Embestida física (Nivel 2)

El ataque en el Nivel 2 es una embestida con física propia: el personaje se desplaza a alta velocidad en línea recta durante una distancia máxima parametrizable, después de la cual entra en un período de recuperación. La velocidad de embestida es ajustable y el agente la incrementa cuando el jugador le está ganando:

$$x(t) = x_0 + \text{velEmbestida} \times \text{dirección} \times t$$

La distancia recorrida se acumula frame a frame; al superarla se activa el período de recuperación, durante el cual el personaje no puede actuar ni recibir nuevas órdenes.

Física 5 — Salto parabólico con gravedad (Nivel 2)

El salto sigue la física de un proyectil bajo gravedad constante. El personaje recibe una velocidad vertical inicial negativa (hacia arriba) y la gravedad la va reduciendo hasta devolver al personaje al suelo:

$$v_y(t) = v_{y_0} + g \times t \quad y(t) = y_0 + v_y(t) \times t$$

Los valores usados son $v_{y_0} = -500$ px/s y $g = 1000$ px/s². Mientras el personaje sube ($v_y < 0$) es inmune a los ataques, lo que crea una mecánica de esquivar mediante salto.

Física 6 — Esquive con impulso amortiguado por fricción (Nivel 2)

El esquive aplica un impulso horizontal brusco al personaje que se desacelera progresivamente por fricción, simulando el esfuerzo físico de un salto lateral rápido sobre terreno volcánico:

$$v_x(\text{frame}+1) = v_x(\text{frame}) \times \text{fricción} \quad (\text{fricción} = 0.85)$$

El movimiento se detiene cuando la velocidad cae por debajo de un umbral (5 px/s), evitando que el personaje se deslice indefinidamente. El impulso inicial es de 1100 px/s, suficiente para desplazarse fuera del rango de la embestida rival.

6.2 Memoria Dinámica

El uso de memoria dinámica es explícito y justificado en todo el proyecto, cumpliendo el requisito del enunciado de gestión manual con new y delete:

- **vector<Roca*>:** las rocas se crean con new al hacer spawn y se liberan con delete dentro del lambda de remove_if. Se usaron punteros crudos en lugar de smart pointers para cumplir el requisito de manejo manual de memoria dinámica.
- **Jugador*, Temblor* en Nivel1:** se crean en el constructor y se destruyen en el destructor con delete. El destructor también itera el vector de rocas liberando cada una.
- **Principal*, Enemigo*, Temblor* en Nivel_2:** mismo patrón. El destructor de Nivel_2 libera los tres punteros.
- **Dificultad* en EscenaNivel1:** se crea al inicio del nivel y se destruye en el destructor de la escena.
- **QMediaPlayer* y QAudioOutput* en SoundManager:** el SoundManager del Nivel 2 crea y destruye sus propios players manualmente.

La regla general aplicada es: quien crea con new es responsable de liberar con delete. Nunca se mezcla la propiedad de punteros entre clases para evitar dobles liberaciones.

6.3 Eficiencia en el Uso de Recursos

Para evitar operaciones costosas en cada frame se aplicaron varias estrategias de caché:

- **Caché de frame de roca:** el vector frameRocaAnterior guarda el último frame de sprite renderizado para cada roca. Solo se llama setPixmap() cuando el frame cambia, evitando copias de QPixmap innecesarias en cada iteración del loop.
- **Caché de HUD:** las variables hudTiempoAnterior, hudRocasAnterior, hudVidasAnterior y hudTemporAnterior almacenan el último valor mostrado. El texto del HUD solo se actualiza cuando el valor cambia, evitando llamadas a setPlainText() en cada frame.
- **Sprites preescalados:** todos los QPixmap se escalan una única vez en cargarSprites() al iniciar el nivel. Durante el juego solo se hace setPixmap() con pixmaps ya procesados.
- **Tipos short:** variables como tiempoRestante, rocasDestruidas, rocasObjetivo y vidas se declaran como short porque sus valores nunca superan 32767, reduciendo el consumo de memoria respecto a int.
- **Referencias constantes:** los métodos getJugador(), getRocas() y getLanza() devuelven const & en lugar de copias, evitando la copia de objetos al acceder al estado del nivel desde la GUI.

6.4 Paso por Referencia y por Puntero

A lo largo del código se aplica consistentemente el principio de evitar copias innecesarias:

- **Paso por referencia constante:** los getters que devuelven objetos grandes usan const&. Por ejemplo, getJugador() const devuelve const Jugador& y getRocas() const devuelve const vector<Roca*>&. Sin el &, cada llamada crearía una copia del objeto completo.
- **Paso por puntero con ownership claro:** Nivel1 recibe Dificultad* sin tomar ownership (el puntero lo crea y destruye EscenaNivel1).

- **Referencias en lambdas:** el lambda de `remove_if` recibe `Roca* r`, que ya es un puntero, evitando copias del objeto `Roca` al iterar el vector.
- **Referencias en funciones de la GUI:** `dibujarHUD(QPainter& p)` recibe el painter por referencia, no por valor, ya que copiarlo no tiene sentido semántico y es costoso.

6.5 Sobrecarga de Constructores

`EscenaTransicion` tiene dos constructores sobrecargados que resuelven dos situaciones distintas con la misma clase:

//transición al inicio: va al nivel

```
EscenaTransicion::EscenaTransicion(MainWindow* ventana, bool esDifícil)
```

//transición al final: va al menú

```
EscenaTransicion::EscenaTransicion(MainWindow* ventana, bool victoria, bool)
```

El tercer parámetro `bool` del segundo constructor es un parámetro dummy que solo existe para diferenciar las firmas en tiempo de compilación, ya que ambos constructores tendrían la misma firma si no estuviera.

6.6 Agente Autónomo

El agente del Nivel 2 (clase `Enemigo`) ejecuta en cada frame un ciclo de cuatro etapas:

- **Percibir:** lee desde `Nivel_2` la distancia al jugador, su última acción, el marcador y si hay temblor activo. Almacena cada acción del jugador en un vector `<Accion>` para el aprendizaje posterior.
- **Razonar:** una máquina de estados finita con seis estados (`ESPERAR`, `AVANZAR`, `ATACAR`, `RETROCEDER`, `SALTAR`, `ESQUIVAR`) con transiciones probabilísticas. Si el jugador ataca, el agente puede esquivar o saltar según probabilidades. Si el jugador está muy cerca, puede retroceder.
- **Actuar:** ejecuta la acción decidida llamando a `mover()`, `atacar()`, `esquivar()` o `saltar()` de `Combatiente` según el estado actual.
- **Aprender:** al cerrar cada punto, analiza el historial de acciones del jugador, actualiza `PerfilJugador` con frecuencias reales (ataques a corto/medio rango, tendencia de esquivar) y ajusta velocidad de embestida, tiempos de espera y distancia preferida de combate.

6.7 Game Loop y Timers

- **Nivel 1:** `QTimer` a 16 ms (`timerLoop`) llama a `actualizarFrame()`. Timers separados controlan el spawn de rocas (`timerSpawn`), los temblores (`timerTemblor`), el countdown de tiempo (`timerSegundo`) y el cooldown de ataque (`timerCooldownAtaque`). Todos se conectan con `connect()` explícito como indica el enunciado.

- **Nivel 2:** QTimer interno de GameWidget a 16 ms llama a gameLoop() que actualiza la lógica y gestiona animaciones y sonidos de forma centralizada.

7. Procedimientos de Prueba

Las pruebas fueron manuales funcionales, verificando cada requerimiento directamente sobre la aplicación compilada.

CP	Caso de prueba	Resultado esperado
01	Navegar del menú al Nivel 1 en Fácil	Se muestra la transición y arranca el nivel en modo fácil.
02	Constructor Roca con velCrecimiento = 0	Lanza invalid_argument.
03	Colisión roca-jugador	El jugador pierde una vida y la roca se desactiva.
04	Victoria del Nivel 1	nivelTerminado() = true, fueExitoso() = true.
05	Temblor en Nivel 1	getOffsetX() y getOffsetY() devuelven valores $\neq 0$ durante 2.5 s.
06	Esquiva del enemigo al ser atacado	El enemigo pasa a estado ESQUIVAR y no recibe daño.
07	Aprendizaje entre rondas	probRetrocederSiCerca aumenta tras repetir ataques cortos.
08	Límites de plataforma en Nivel 2	El personaje queda reubicado al borde sin errores.
09	Volver al menú desde el Nivel 2	juegoTerminado() se emite y MainWindow regresa al menú.
10	Constructor Temblor con omega = 0	Lanza invalid_argument.
11	Pulsar P durante el Nivel 1	Todos los timers se detienen y se regresa al menú.
12	Dificultad Difícil vs Fácil	Las rocas crecen más rápido y el temblor es más fuerte.

Todos los casos obtuvieron el resultado esperado en la revisión final previa a la entrega.

8. Guía de Instalación y Uso

8.1 Compilación desde código fuente

Abrir el archivo esgrimaNivel1.pro en Qt Creator y pulsar Run.

8.3 Ejecución del ejecutable

Si se usa el ejecutable precompilado, descomprimir la carpeta y ejecutar esgrimaNivel1.exe

8.4 Manual de Controles

NIVEL 1 — (Vista cenital)

Tecla	Acción
W	Mover hacia arriba
S	Mover hacia abajo
A	Mover hacia la izquierda
D	Mover hacia la derecha
Espacio	Atacar: golpe con la lanza en la dirección que mira el personaje
P	Volver al menú principal

NIVEL 2 — (Vista lateral)

Tecla	Acción
A	Mover hacia la izquierda
D	Mover hacia la derecha
W	Saltar
Shift	Esquivar: impulso lateral rápido (inmune durante el movimiento)
Espacio	Embestir: ataque con la lanza hacia adelante
P	Volver al menú principal

9. Resultados y Discusión

9.1 Funcionamiento Logrado

- Ambos niveles son jugables con mecánicas diferenciadas y transiciones fluidas entre ellos.
- El agente autónomo reacciona al comportamiento del jugador y ajusta su dificultad de forma visible entre rondas.
- La física del temblor sísmico funciona consistentemente en QGraphicsScene (Nivel 1) y QWidget (Nivel 2).
- El sistema de dificultad modifica seis parámetros independientes de forma efectiva sin alterar la lógica central.

- El audio está integrado en ambos niveles con música de fondo y sonidos de eventos.

9.2 Limitaciones Actuales

- El estado aprendido del agente no persiste entre sesiones: al cerrar el juego el agente vuelve a su comportamiento inicial.
- No existe pantalla de pausa real: presionar P regresa directamente al menú sin opción de continuar.
- Interfaz gráfica mejorable.

9.3 Mejoras Planeadas

- Agregar persistencia del perfil del agente en archivo JSON entre sesiones.
- Implementar pantalla de pausa con opción de continuar o reiniciar.
- Agregar más niveles de dificultad aprovechando el patrón Strategy ya implementado.
- Mejorar la interfaz grafica en todo el programa.

10. Conclusiones

El desarrollo del juego demostró que es posible construir un videojuego funcional aplicando de forma coherente los conceptos del curso: jerarquías de herencia propias, polimorfismo, patrones de diseño, separación entre lógica y presentación, manejo explícito de memoria dinámica y un agente autónomo con aprendizaje adaptativo.

La decisión más importante del proyecto fue mantener la lógica del juego completamente independiente de Qt. Esto hizo posible que los dos niveles, desarrollados en ramas separadas por distintos integrantes, pudieran integrarse sin grandes conflictos. La clase Temblor compartida entre ambos niveles es el ejemplo más claro de ese beneficio: una sola implementación sirve para dos contextos de renderizado completamente distintos.

El agente autónomo del Nivel 2 cumple las cuatro propiedades requeridas: percibe el estado del mundo, razona mediante una máquina de estados con transiciones probabilísticas, actúa sobre el mundo a través de los métodos de Combatiente, y aprende actualizando su perfil y parámetros de comportamiento tras cada punto disputado.

Como aprendizaje del proceso, planificar la integración desde el inicio habría reducido los ajustes necesarios al unificar ambas ramas, especialmente en la clase Temblor y la arquitectura de MainWindow. La planificación anticipada de los puntos de integración es tan importante como el desarrollo de las funcionalidades individuales.